ProCabinet.app

Strategic Plan Review

A scannable, decision-ready snapshot of PLAN.md

Snapshot date: 2026-05-05

STATUS

App live at procabinet.app — Cloudflare Pages, auto-deploy on push to main.

Pre-launch refactor complete (SPEC.md Phases 0–7, TypeScript strict mode).

~9 days to mid-May 2026 launch target.

PLAN.md remains the live working document. This is a strategic snapshot for prioritisation conversations.

1. Executive Summary

CRITICAL

The aim

ProCabinet.app is the only integrated cloud platform for small cabinet shops — combining cut lists, quoting, order management, stock tracking, and scheduling at $29/mo, undercutting Mozaik ($50–95) and Cabinet Vision ($5,000+).

Target users are 1–20-person joinery and cabinet shops currently bouncing between spreadsheets, paper, and disconnected tools — a long-tail market underserved by enterprise CAD and over-served by basic spreadsheets.

The Year-1 plan targets 280 Pro subscribers (~$97k ARR) by month 12, with a 10–15% free→Pro conversion target and CAC under $15 — meaning organic channels (Reddit, forums, Product Hunt, SEO) are the primary acquisition strategy.

The big strategic call

Speed to market beats feature completeness.

The free tier is the growth lever; conversion happens at limit boundaries, not at signup. The fastest path to validating the business model is: ship Stripe + free-tier limits + a landing page, then learn from the first 50 signups. Polish, mobile, and refactor work should be ruthlessly deferred unless they directly unlock revenue.

Top 5 recommendations

RECOMMENDATION

1. Resolve the 5 open decisions today (D.1–D.5) — they're each blocking specific sub-step planning. See § 4.

2. Ship Stripe + free-tier limits + landing page first — in that order. Nothing else unblocks revenue. See § 5.

3. Defer Item 2 Phase 4 cleanup until post-launch — internal-only refactor with zero user impact. Phases 1 + 2 + 3 already shipped; only Phase 4 remains. See § 6.

4. Don't touch the refactor backlog pre-launch — cabinet.js / cutlist.js splits, naming cleanup, etc. Zero user value before launch.

5. Cut launch-week scope to 3 channels — r/woodworking + Product Hunt + waitlist email on day 1; HN/IndieHackers/forums staggered through week 2.

2. How to Read This Document

INFO

Importance ratings

CRITICAL	Blocks revenue, launch, or both. Do now.
IMPORTANT	Visible quality matters; ship before launch in scoped form.
SHOULD-HAVE	Improves user experience; targeted pre-launch, full pass post-launch.
NICE-TO-HAVE	Real value but no user-visible impact pre-launch.
DEFER	Don't touch until post-launch unless something forces it.
DONE	Already shipped — listed for context, no action needed.

Section structure

Every actionable section follows the same pattern: Why it matters → Scope → Pros & Cons (where there are tradeoffs) → Recommendation (callout box).

IDs in monospace (e.g. `S.4`, `F.2`, `D.1`) cross-reference items in PLAN.md. Open it side-by-side to drill into sub-steps.

3. Status at a Glance

CRITICAL

Twelve work areas, importance-ranked. Anything red is currently blocking revenue or launch; anything grey can wait. Item 2 Phase 2 just shipped (2026-05-05) — moved to DONE.

Area	Status	Importance	Recommendation
Live deployment	Done	DONE	Maintain.
Stripe + subscriptions	Not started	CRITICAL	Start now. § 5a.
Free-tier limits	Not started	CRITICAL	Start after Stripe S.3. § 5b.
Landing page	Not started	CRITICAL	Resolve D.2, then build. § 5c.
Onboarding	Not started	IMPORTANT	Lean version pre-launch. § 5d.
Item 2 Phase 2 (UI unification)	Done 2026-05-05	DONE	Edit/Duplicate/Create-Quote shipped. § 6a.
Item 2 Phase 4 (cleanup)	Pending	NICE-TO-HAVE	Defer to post-launch. § 6b.
Production ops	Mostly missing	IMPORTANT	Minimum viable: P.3 (errors) only. § 7.
Mobile + responsive	Not started	SHOULD-HAVE	iPad-landscape pass before launch. § 9.
UI polish	Not started	SHOULD-HAVE	Targeted to activation funnel only. § 10.
Refactor backlog	Deferred	DEFER	Do not touch pre-launch. § 11.
Launch week execution	Not started	CRITICAL	Cut to 3 channels day 1. § 12.

4. Open Decisions (resolve first)

CRITICAL

Five decisions are blocking specific sub-step planning below. Each is small in scope but compounds: the longer they stay open, the more sub-steps stall waiting for them.

D.1 — Free-tier enforcement style

When a free-tier user hits a limit (e.g. 21st part, 6th active quote), do we soft-block (allow with upgrade prompt) or hard-block (refuse the action)?

Option A — Soft block everywhere

PROS	CONS
+ Preserves activation; user keeps building, sees value	– Weaker conversion signal — user may never feel pressure to upgrade
+ Lower drop-off in first session	– Free tier becomes "infinitely usable", undermining $29 value prop
+ Feels more generous, builds goodwill	– Harder to tie KPI to specific action moment

Option B — Hard block everywhere

PROS	CONS
+ Clearest Pro value prop ("you literally cannot continue without paying")	– High first-session abandonment risk
+ Forcing function for the 10–15% conversion target	– Can feel hostile; bad first impression on Reddit/PH launches

RECOMMENDATION

Hybrid (recommended): soft for parts and clients (preserve activation), hard for active quotes and orders (the moments where Pro value is clearest).

Rationale: protects the activation funnel — first cut list and client setup are the "wow" moments — while creating real conversion pressure exactly where the user is committing to a paying customer (a quote → order).

D.2 — Landing page placement

Currently the app lives at procabinet.app root. To add a marketing landing page (hero, pricing, signup CTA), where should it go relative to the app?

Option A — App stays at root, landing at /marketing or replaces root

PROS	CONS
+ No DNS/auth changes	– Auth redirect URLs need updating if root becomes marketing
+ Familiar to existing users	– Mixed concerns at root (marketing + app)
	– SEO complications for ranking the marketing page

Option B — Subdomain split (app.procabinet.app + procabinet.app for marketing)

PROS	CONS
+ Standard SaaS convention; users expect this	– Requires DNS record + Cloudflare Pages second project
+ Clean separation of concerns	– All auth redirect URLs in Supabase need updating
+ Marketing page can rank for SEO independently	– Existing bookmarks break (one-time cost)
+ Easy to swap marketing tech later (move to Framer / Webflow without touching app)	

Option C — Path split (/app for app, / for marketing)

PROS	CONS
+ Single domain, single SSL cert, simpler hosting	– All app paths shift; auth URLs need updating
+ No DNS work	– More client-side routing complexity
	– Less standard for SaaS, harder to swap marketing later

RECOMMENDATION

Option B (subdomain split). Aligns with SaaS norms, lets marketing tech evolve independently of app, and the auth-URL update is a one-time chore.

Migration order: build marketing page on procabinet.app (direct upload to Cloudflare Pages), then move app to app.procabinet.app and update Supabase auth URLs in one cutover.

D.3 — Path C: cabinet templates vs instances

The cabinet_templates DB table currently doubles as the saved-cabinet library; cabinet instances inside a project are written to quote_lines. This works but conflates two concepts.

Option A — Keep current setup (templates table = library; instances = quote_lines)

PROS	CONS
+ Zero work; ships today	– Conceptual confusion in code (template vs instance not separable)
+ No risk of regression	– Limits future features like "apply template to existing project"

Option B — Promote cabinets table (already exists, currently unused)

PROS	CONS
+ Clean separation: templates = library, cabinets = project instances	– Significant migration work (cabinet.js largely rewritten)
+ Aligns with SCHEMA.md original design	– Pre-launch risk; nothing user-facing changes

Option C — Unify (one cabinet_designs table with is_template flag + optional project_id)

PROS	CONS
+ Most elegant long-term	– Highest risk migration
+ Single CRUD path for everything	– Doesn’t solve any user problem visible today

RECOMMENDATION

Option A — keep current setup. Defer Path C indefinitely. Item 2 Phase 4 cleanup is touching this surface; revisit only if Phase 4 surfaces a concrete user-facing problem the current design blocks.

Strategic rationale: zero user-visible benefit, zero revenue impact. Spend the cycles on Stripe instead.

D.4 — Analytics provider

Plausible Analytics ($9/mo)

PROS	CONS
+ Privacy-friendly; no cookie banner needed	– $9/mo recurring cost
+ Lightweight script (<1KB)	– Smaller ecosystem, fewer integrations
+ Better suits B2B SaaS positioning	
+ GDPR-compliant by default	

Google Analytics (free)

PROS	CONS
+ Free	– Cookie banner required (GDPR)
+ Massive ecosystem, mature tooling	– Heavier script; affects page-load metric
+ Familiar to most developers	– Conflicts with privacy-friendly brand positioning
	– Data harvested for ad ecosystem

RECOMMENDATION

Plausible. Cabinet shop owners are not ad-targeted demographics; the privacy story doubles as marketing. The $9/mo is rounding error vs Pro MRR targets.

D.5 — Error logging provider

Sentry (5k events/mo free)

PROS	CONS
+ Best-in-class client-side error capture	– Heavier SDK
+ Source map support, breadcrumbs, replays	– Free tier may not cover post-launch surge
+ Free tier covers a small SaaS comfortably	

Better Stack / Logflare

PROS	CONS
+ Combined logs + uptime	– Less mature client-side error UX
+ Cheaper at scale	– More setup required

RECOMMENDATION

Sentry. Free tier is sufficient for the first 50–500 active users; the source-map and replay UX is unmatched for diagnosing client-side bugs in beta. Reassess if you blow through the free tier.

5. Critical Path to Launch

CRITICAL

Four areas in dependency order. Stripe → Free-tier limits → Landing page → Onboarding. Nothing else unblocks revenue.

5a. Stripe + Subscriptions

Why it matters: Revenue gate. Without Stripe, no path from free → Pro and the business model can't validate. Every day without it is a day of free tier with no upgrade option for users who want to pay.

Scope: PLAN.md items `S.1` – `S.8` — account, products, schema, Checkout, portal, webhooks, invoices, e2e test.

Stripe vs Paddle / LemonSqueezy

PROS	CONS
+ Lowest fees (2.9% + 30¢ vs 5–8% for merchant of record)	– You handle tax compliance (vs Paddle/LS = merchant of record)
+ Most mature webhook ecosystem	– Need to register for sales tax in target jurisdictions eventually
+ Customer Portal eliminates need to build upgrade/cancel UI	
+ Already in plan	

RECOMMENDATION

Stay with Stripe. Use Customer Portal for all subscription management UI — the time savings vs custom-building outweighs the slightly less polished UX. Defer tax-compliance complexity until first 50 paying customers.

Critical sub-steps: `S.3` (subscriptions table) before `S.4` (Checkout); `S.6` (webhooks) before `S.8` (e2e). Skip `S.7` (custom invoice views) for v1 — Customer Portal includes invoice history.

5b. Free-Tier Limits

Why it matters: The free tier is the growth lever — limits are what create the conversion prompt. Without them, you have no funnel.

Scope: PLAN.md items `F.1` – `F.5`. Implementation depends on D.1 (soft vs hard).

RECOMMENDATION

Ship the hybrid enforcement model (per D.1 recommendation): soft for parts and clients, hard for active quotes and orders.

Add limits incrementally: parts first (most-touched limit), then quotes, then orders, then stock and clients. Each one is one PR; smoke test after each.

Instrument from day 1: log every limit-prompt impression to Plausible (D.4) so you can measure conversion at each boundary and tune the limits.

5c. Landing Page

Why it matters: Conversion entry point. The current root URL drops cold visitors directly into the app login screen — no value prop, no pricing, no trust signals. Every Reddit/PH visitor sees this.

Scope: PLAN.md items `C.1` – `C.7`. Placement depends on D.2 (subdomain vs path).

RECOMMENDATION

Ship lean: hero + features + pricing + email-capture-or-signup CTA only. Skip testimonials (will be empty), demo video (defer to launch week +14), blog posts (defer 1 of 3 minimum).

Build on Cloudflare Pages as a separate static project (HTML + Tailwind via CDN, no build step). Single page; iterate post-launch.

Include a "What's ProCabinet?" 30-second pitch above the fold — assume zero context, since organic traffic from Reddit will be cold.

5d. Onboarding

Why it matters: High churn risk after first session. Activation rate (% who create a project) is the leading indicator of revenue — Business Plan targets 60%.

RECOMMENDATION

Lean version only: a single welcome modal on first sign-in + a pre-loaded sample project (one client, one cut list with 5 sample pieces, ready to optimise).

Defer to v1.1 (post-launch): coachmarks/tooltips, branded email templates, multi-step tour, video embeds.

The sample project alone should drive activation; real users will explore from a working template faster than from an empty state.

6. Active Refactor — Item 2 (Cabinet/Quote Unification)

IMPORTANT

Phases 1 + 2 + 3 done. Only Phase 4 (cleanup) remains — and it's deferrable.

6a. Phase 2 — UI Unification ✅ Done 2026-05-05

Status: DONE — shipped end-to-end.

All five sub-steps landed: `2.1` project-required guard with auto-create, `2.2` in-place quote editing via `editQuoteInCB` (with persistent editing context across refreshes), `2.3` Duplicate button (already existed via `duplicateQuote`), `2.4` single "Create Quote" button via `cbCreateQuoteFromDraft` (replaces the old new/existing pair), `2.5` verified clean via browser eval and `npm run typecheck`.

Net result: the cabinet → quote workflow is now a single editing surface. Beta testers won't surface the rough edges that originally motivated this phase.

6b. Phase 4 — Cleanup

Importance: NICE-TO-HAVE — defer to post-launch.

Internal-only: dead code removal (cbSavedQuotes, cbProjectLibrary), drop converter pair, optional file rename. Zero user-visible impact.

RECOMMENDATION

Defer all of Phase 4 (4.1–4.5) to post-launch v1.1. The dead code isn't hurting anything and removing it carries small regression risk for zero user benefit.

Exception: 4.1 (catalog_items CRUD wiring) is a real bug — material/handle/finish edits don't persist. If a beta tester reports it, escalate to pre-launch. Otherwise it can wait.

7. Production Readiness

IMPORTANT

Five sub-items in PLAN.md (`P.1` – `P.5`). The temptation is to do all of them; the launch-velocity move is to ship only what catches bugs, defer what catches process issues.

Full production setup pre-launch

PROS	CONS
+ Maximum confidence on launch day	– Adds 3–5 days to timeline
+ Catches issues before users see them	– Most issues only surface with real users anyway
	– Cross-browser sweep without users is busywork

Minimum viable: Sentry only

PROS	CONS
+ Ships in hours, not days	– Some launch-day bugs will be discovered live
+ Captures real user errors as they happen	– No "clean cutover" — production env stays mixed
+ Other items fold naturally into beta week	

RECOMMENDATION

Minimum viable for launch: ship P.3 (Sentry) only. Defer P.1, P.2, P.4, P.5 into beta week as they're needed.

Sentry catches the bugs that matter (real user errors); cross-browser sweeps and full e2e tests catch issues that may never bite — pay that cost when you have signal that they actually happen.

Exception: if you spin up a separate prod Supabase project (P.1), schema changes get safer post-launch — tradeoff worth considering if you anticipate frequent schema iteration.

8. Pre-Launch Content

IMPORTANT

PLAN.md items `C.1` – `C.7`. The whole list is a nice-to-have except landing page (already CRITICAL in § 5c). Be ruthless about what ships day 1 vs week 2 vs month 2.

Item	When	Notes
Landing page (C.2)	LAUNCH DAY	Single page, hero + pricing + email/signup
Email waitlist capture (C.2)	LAUNCH DAY	Embed in landing page; collect from day 1
1 SEO blog post (C.4)	LAUNCH DAY	Pick the highest-intent: "How to price cabinet jobs"
Plausible + Search Console	LAUNCH DAY	Need analytics live before launch traffic
Beta outreach (C.6)	LAUNCH WEEK	10 cabinet makers, personal asks
Launch announcement copy	LAUNCH WEEK	Reddit + PH + email; cut HN/IH for week 2
2 more SEO blog posts	WEEK +14	Cut list software, plywood waste
Demo video (C.3)	WEEK +14	2 min Loom; defer until you have a stable workflow to demo
Testimonials	MONTH 2	Wait for beta feedback; placeholder is worse than nothing

RECOMMENDATION

Launch day = 4 things only. Landing page, email capture, 1 blog post, analytics. Everything else can ship in week 2 or later.

Better to launch with less polish than to delay 5 days for week-3 content. Reddit doesn't care about your testimonials section; they care if the cut-list tool actually works.

9. Mobile + Responsive

SHOULD-HAVE

PLAN.md items `M.1` – `M.5`. Workshop reality: cabinet shops use phones for on-site quoting, tablets in the workshop, but desktop is primary for cut-list optimisation (the killer feature).

Full mobile pass before launch

PROS	CONS
+ Best first impression on mobile-first traffic	– Cabinet Builder + Cut List have complex layouts that resist quick responsive fixes
+ Avoids "unusable on phone" Reddit comments	– 5–10 days of work
	– Phone usage is secondary to desktop for the core workflow

iPad-landscape pass only (recommended)

PROS	CONS
+ Workshops use iPads; landscape is the actual primary mobile use case	– Phone users still see broken layouts
+ Most layouts already work at iPad-landscape width with minor tweaks	– Need clear "best on desktop" messaging on landing page
+ 1–2 days of work	

RECOMMENDATION

iPad-landscape pass only before launch. Cabinet Builder, Cut List, Quotes — verify they work at 1024×768.

Add a friendly "Best experienced on desktop or tablet" banner on phones; full phone responsive in v1.1.

Skip PWA manifest pre-launch — installing as an app makes more sense after users have committed to using the product daily.

10. UI Polish

SHOULD-HAVE

PLAN.md items `U.1` – `U.7`. Polish has diminishing returns; the activation funnel matters most.

RECOMMENDATION

Targeted polish only. Focus pre-launch effort on: signup flow, first-cut-list experience, paywall modal, and Pro upgrade CTA. These are the screens beta testers see at the highest-impact moments.

Defer everything else: design system documentation, full empty/loading/error state audit, accessibility deep-dive, full user testing. Pick these up when beta feedback identifies specific friction.

Do verify: every async action shows a loading state, every error throws a user-facing toast (silent failure is worse than ugly errors).

11. Refactor Backlog

DEFER

PLAN.md items `R.1` – `R.4`. Pure technical debt with zero user impact — split cabinet.js (2,543 lines), split cutlist.js (2,946 lines), shadow-name unification, relocate stragglers.

RECOMMENDATION

Do not touch pre-launch. The 1500-line target from SPEC.md is a quality-of-life metric for the developer, not a user-facing one. The files work; splitting them is paperwork.

Pick up R.1 / R.2 only when cabinet.js or cutlist.js becomes painful to edit (e.g. AI-assisted edits start failing due to file size, or you can't hold the file in your head).

R.3 / R.4 should be done opportunistically — alongside any feature work that touches those files, not as standalone tasks.

12. Launch Week

CRITICAL

PLAN.md items `L.1`–`L.4`. Channel discipline matters more than channel volume — launching on too many channels at once means none gets focused effort.

When	Channel	Effort
LAUNCH DAY	r/woodworking — well-crafted post, demo GIF, transparent about being early	High
LAUNCH DAY	Product Hunt — Tuesday for best traffic; have a maker-comment thread ready	High
LAUNCH DAY	Email to waitlist subscribers	Low
DAY +3	r/cabinetmaking (smaller, more targeted than r/woodworking)	Medium
DAY +5	Show HN — only after landing page polish from launch-day feedback	Medium
DAY +7	IndieHackers — different audience, less risk if it underperforms	Low
WEEK 2	Facebook cabinet-making groups (drip-fed, 1 group/day)	Medium
WEEK 2	SawmillCreek + WoodworkingTalk (text posts, not link-drops)	Medium

RECOMMENDATION

Launch day = 3 channels (r/woodworking + Product Hunt + waitlist email). Each gets a tailored post; no copy-paste.

Stagger HN, IH, r/cabinetmaking, and forum posts across days 3–7. This gives time to fix bugs surfaced by initial traffic, and avoids the "spammy launch" pattern.

Watch r/woodworking for tone — over-promotional posts get downvoted. Lead with a screenshot of the cut-list optimiser, not a sales pitch.

13. Post-Launch Growth (Months 2–12)

INFO

PLAN.md items `G.1` – `G.8`. Not actionable yet; included for visibility. The plan calls for weekly SEO posts, monthly YouTube tutorials, supplier outreach, and a referral programme.

RECOMMENDATION

Don't plan in detail until first 50 signups land. The data will reshape priorities — what people churn on, what features get requested, which acquisition channel actually works — and pre-planning the first 6 months of growth is wasted effort.

Do commit to the cadence framing: weekly content + monthly video + monthly feature release. Specifics emerge from real user signal.

14. Milestones + KPIs

INFO

Date	Target
May 2026	50 signups in first week
June 2026	First 5 paying Pro subscribers ($145 MRR)
August 2026	200 free / 25 Pro ($725 MRR)
November 2026	Production scheduling module live (Pro exclusive)
February 2027	1,000+ free / 140+ Pro ($4,000+ MRR)
May 2027	Year 1 anniversary — Shop tier launched (multi-user)

KPI targets

- Signup → active (creates first project): **60%**
- Free → Pro within 30 days: **10–15%**
- Monthly churn: **under 5%**
- CAC (blended): **under $15**

RECOMMENDATION

Instrument the activation funnel from day 1: signup, sample-project completion, first-real-project, first-quote, first-order, upgrade-prompt-shown, upgrade-completed.

These four KPIs should be on a dashboard you check daily for the first 60 days. Plausible (D.4) handles top-of-funnel; Stripe Dashboard handles conversion + churn.

15. Resolved / Historical (no action)

INFO

Tracked here so the strategic snapshot is self-contained. None of these items require work — they document deviations from SPEC.md and implicit resolutions that closed during the pre-launch refactor. Full history in `SPEC.md § 13`.

SPEC § 7 success criteria — deviations accepted

File-size target (<1500 lines per file) — partially met. `src/cabinet.js` (~2,543) and `src/cutlist.js` (~2,946) exceed the target. Tracked as actionable items `R.1` + `R.2` in PLAN.md's refactor backlog; the deviation itself is accepted indefinitely as zero-user-impact debt.

`<script type="module">` migration — deferred indefinitely. Original SPEC called for ES modules in `index.html`; shipped instead with classic `<script defer>` tags loading the carved files. ES-module conversion would require ~80–100 `window.X` shims for inline event handlers across 411 attributes — low-value churn vs zero user benefit. Decision in SPEC § 13 2026-04-28 ("Phase 6 PARTIAL").

SPEC § 9 open questions — implicitly resolved

SPEC § 9 question	Resolution
Cabinet systems unification (now or post-launch)	Done as Item 2 — Phases 1 + 2 + 3 complete; Phase 4 cleanup remaining.
Catalog storage (per-user tables vs jsonb)	Unified catalog_items table with type column (material / handle / finish / hardware).
Logo storage	Supabase Storage at business-assets/{user_id}/logo.{ext} (Phase 3.3).
Quote sources (cbSavedQuotes LS or DB)	DB sole source of truth. cbSavedQuotes removal still tracked as Phase 4.2.
Vite (in scope or post-launch)	In scope, shipped (Item 3 phases A/B/D, 2026-04-30).
Types (JSDoc / TS / neither)	JSDoc + checkJs:true + strict:true across all 19 src files. .ts rename in Backlog → Deferred.
Migration approach (big-bang or phased)	Phased. Schema → migration code → reads → CSS extract → JS extract → module split → cleanup.
Existing data preservation	One-shot migrateLocalToDB() in src/migrate.js, idempotent, exposed via Settings → Backup & Migration.
Smoke test list	Written as SMOKE_TESTS.md. Referenced from P.4 / P.5.

Other historical resolutions

Path B chosen for cabinet templates vs instances (architecture-level). `cabinet_templates` backs the saved-cabinet library; instances inside projects are written to `quote_lines`. Path C (full unification with `is_template` flag) tracked separately as `D.3` if a future user-facing problem makes it worth the migration.

`cq*` → `cb*` symbol-prefix rename — complete. Verified by `grep -rE "\bcq[A-Z]" src/index.html` returning zero matches (2026-05-05). Old `pc_cq_*` localStorage keys are still referenced — intentionally — in `src/migrate.js` to read legacy data during migration; their removal is part of Phase 4.3.

GitHub Actions Node-24 deadline (June 2026). Already opted in via `lts/*` (commit `3417b7f`) and explicit Node 24 (commit `e1ecd75`). Tracked in Backlog → Housekeeping `H.4`; no further action expected.

16. Appendix

INFO

Stack

Layer	Choice	Status
Frontend	Vite + plain HTML/CSS/JS (no framework), 19 src files	DONE
Type-checking	TypeScript strict mode via JSDoc + checkJs:true	DONE
Auth + Database	Supabase (Postgres + RLS, project mhzneruvlfmhnsohfrdo)	DONE
Hosting	Cloudflare Pages, auto-deploy via GitHub Actions	DONE
Domain	procabinet.app (Cloudflare DNS, Bot Fight on)	DONE
Storage	Supabase Storage (business-assets bucket)	DONE
Payments	Stripe	CRITICAL
Email	Supabase auth defaults (needs branding)	IMPORTANT
Analytics	Plausible (per D.4)	CRITICAL
Error logging	Sentry (per D.5)	IMPORTANT

Monthly cost (estimated)

Service	Cost
Cloudflare Pages	Free
Supabase	Free tier (until ~10k MAU)
Stripe	Free (2.9% per transaction)
Domain (procabinet.app via IONOS)	~$1/mo
Plausible Analytics	$9/mo
Sentry (free tier)	Free
Total	~$10/mo

Companion documents

- **PLAN.md** — live working document with full sub-step detail
- **SPEC.md § 13** — full decision log of the pre-launch refactor
- **SCHEMA.md** — complete database schema
- **SMOKE_TESTS.md** — manual smoke-test catalogue
- **Building Docs/ProCabinet_Business_Plan.docx** — strategic context
- **Building Docs/ProCabinet_ToDo_List.docx** — original launch checklist
- **Building Docs/ProCabinet_Outstanding_Features.docx** — original outstanding features list
- **Building Docs/Cabinet_Builder_Guide.docx** — Cabinet Builder technical reference